Project Proposal document

Project Overview:

Summary: An interactive culinary explorer is a web application designed to help users discover meals based on the amount of time it takes to cook, the ingredients. Users will also be able to save recipes. It will use a public culinary API to get recipe data and present it with a clean interface.

The purpose is to reduce the meal planning hassle, reduce food waste by recommending recipes based on what the user already has. The target users are home cooks, people who don't have enough time to spend a lot of time cooking, students, and people who are looking to be healthier.

Key Features:

Search and Filter: Users will be able to search recipes with key words, ingredients or a meal category like breakfast, lunch and dinner.

Recipe Details Modal: users will be able to see detailed cooking instructions, ingredient checklists and a nutrition breakdown.

Favorites: Users will be able to mark recipes down as favorites, so they don't have to go searching for the recipes they like every time they want to cook it.

Custom Meal Planner: The user will be able to add recipes to a daily meal schedule and calculate the total nutritional value.

JavaScript usage:

Async fetching will allow the user to load recipes from an external API without reloading the page.

Dynamic DOM Updating: This will allow the user to programmatically create, modify and clear recipe cars, modals and collections.

Interactive event handling will allow the user to change what they search, select and remove favorites, and modal toggles.

CSS:

A styles sheet will provide a responsive layout that will be accessible on desktop, mobile and tablets. Subtle details like shadow elevation, hover micro-interactions, and animated loading skeletons. The website will also be able to be used in dark mode and light mode.

Technologies used:

Variables, Data Types & Control Flow: Scope management using const, and let, array and object manipulation using map, filter, reduce, and error-handling conditions.

Functions & Objects: Helper functions, arrow functions, destructured objects, and utility methods.

DOM Manipulation & Event Handling: Efficient event delegation on dynamic list containers, dynamic node creation, and CSS class toggling.

API Calls & Async JavaScript: Native fetch () API with async/ await handling HTTP status codes, query formatting, and parsing JSON payloads.

Object Oriented Programming:

Recipe: class representing individual recipe instances, handling formatting, bookmark status, and ingredient scaling.

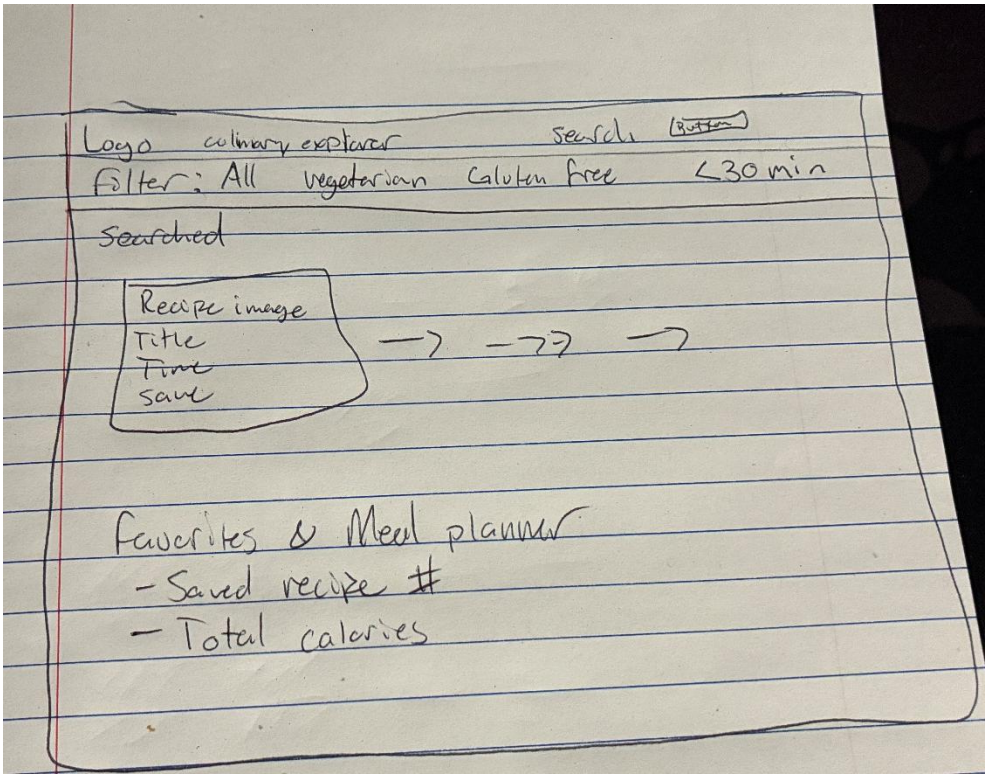
MealPlanner: Class managing daily schedules, adding and removing recipes, and calculating the total calories and macros for meals.

RecipeAPI: class encapsulation API endpoints, rate-limit logic, and request formatting.

UI: class handing element selection, template rendering, and modal states.

LocalStorage Management: JSON serialization (JSON.stringify/JSON.parse) to persist favorite recipes and user planning state across sessions.

UI Sketch:



PHASE 1: Setup & DOM Foundation. Base HTML skeleton, structural CSS layout and UI element bindings. Days 1–2

PHASE 2: OOP & API Integration. Implement RecipeAPI and Recipe classes. Integrate fetch () requests and handle asynchronous data parsing. Days 3–4

PHASE 3: Interactive UI & Rendering Build dynamic card rendering, search listeners, event delegation, and recipe modal views. Days 5–6

PHASE 4: Planner & Local Storage. Build MealPlanner OOP logic, favorites saving/loading using localStorage, and calorie accumulation. Days 7–8

PHASE 5: CSS Refinement & PolishUI micro-interactions, responsive media queries, accessibility checks, and error feedback states. Days 9–10